

LABORATORY WORK #3 (SABIEV NURDAULET 25B030236)

Data Manipulation Language (DML) Operations

1. Introduction

In this lab work, we continue working with the relational Airport Database, which was created and normalized in Lab 2. The main goal is to practice Data Manipulation Language (DML) using PostgreSQL. We will use commands like INSERT, UPDATE, and DELETE. We will also learn how to create dummy data using generate_series(), work with text and dates, and use the RETURNING keyword to see our results immediately.

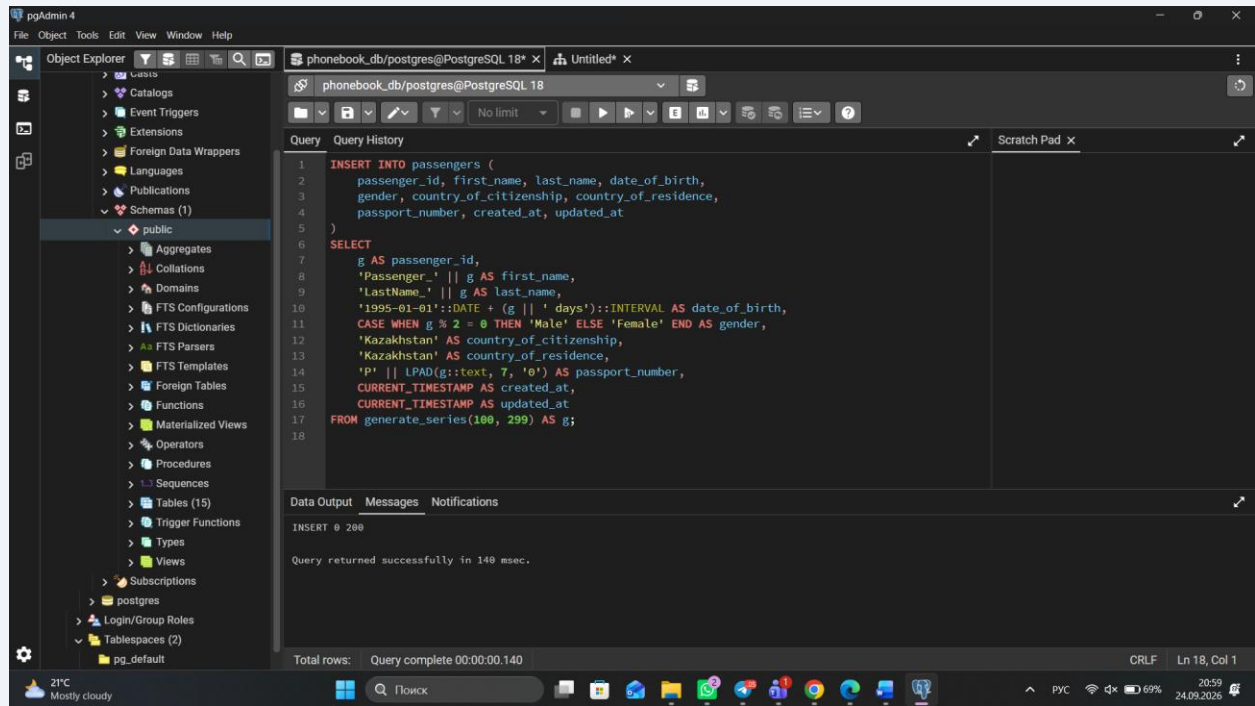
2. Tasks & DML Implementations

Task 1: Generate and insert 200 random rows

Description: Create 200 new passenger records automatically using the PostgreSQL generate_series() function.

```
INSERT INTO passengers (  
    passenger_id, first_name, last_name, date_of_birth,  
    gender, country_of_citizenship, country_of_residence,  
    passport_number, created_at, updated_at  
)  
SELECT  
    g AS passenger_id,  
    'Passenger_' || g AS first_name,  
    'LastName_' || g AS last_name,  
    '1995-01-01'::DATE + (g || ' days')::INTERVAL AS date_of_birth,  
    CASE WHEN g % 2 = 0 THEN 'Male' ELSE 'Female' END AS gender,  
    'Kazakhstan' AS country_of_citizenship,  
    'Kazakhstan' AS country_of_residence,  
    'P' || LPAD(g::text, 7, '0') AS passport_number,  
    CURRENT_TIMESTAMP AS created_at,  
    CURRENT_TIMESTAMP AS updated_at  
FROM generate_series(100, 299) AS g;
```

Explanation: The generate_series(100, 299) function generates 200 numbers. We use these numbers (g) to build fake names, passport numbers, and calculate birthdays. This allows us to insert 200 rows with a single query.



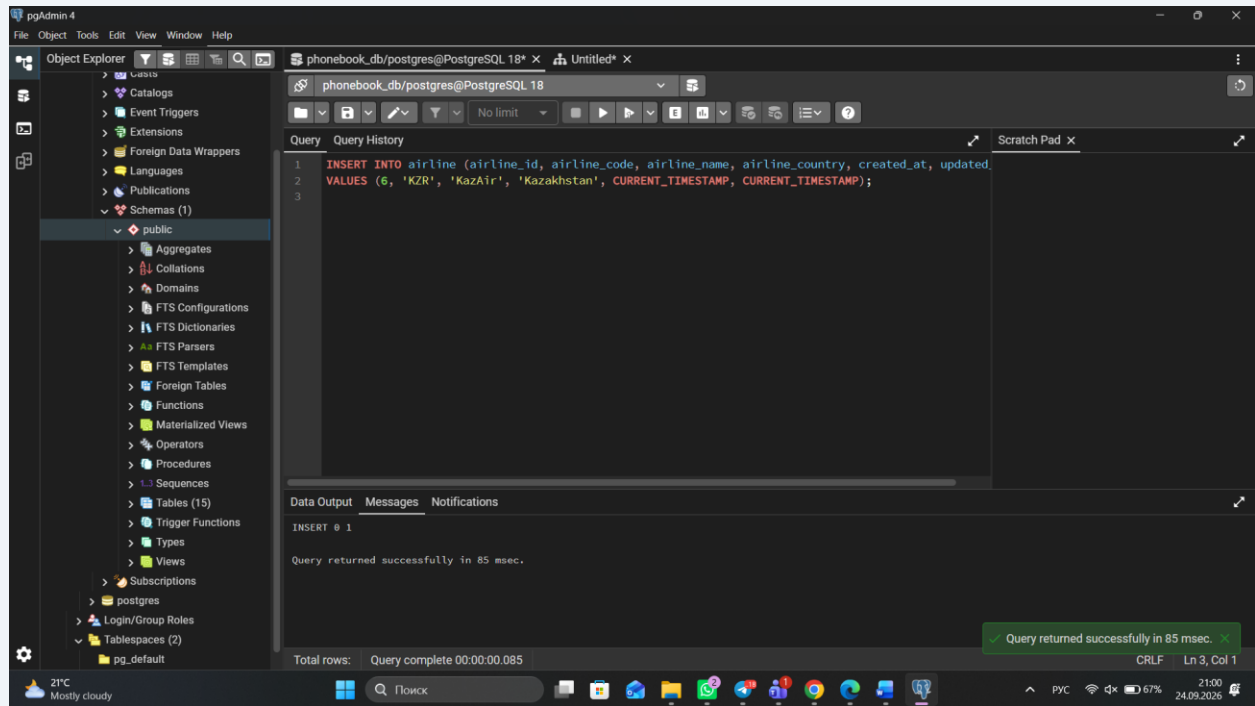
TASK1

Task 2: Add a new airline 'KazAir'

Description: Insert an airline named 'KazAir' from 'Kazakhstan' into the airline table.

```
INSERT INTO airline (airline_id, airline_code, airline_name, airline_country,
created_at, updated_at)
VALUES (6, 'KZR', 'KazAir', 'Kazakhstan', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
```

Explanation: The INSERT INTO command adds one new row to the table. We manually assign the ID 6, the country, the name, and the current time.



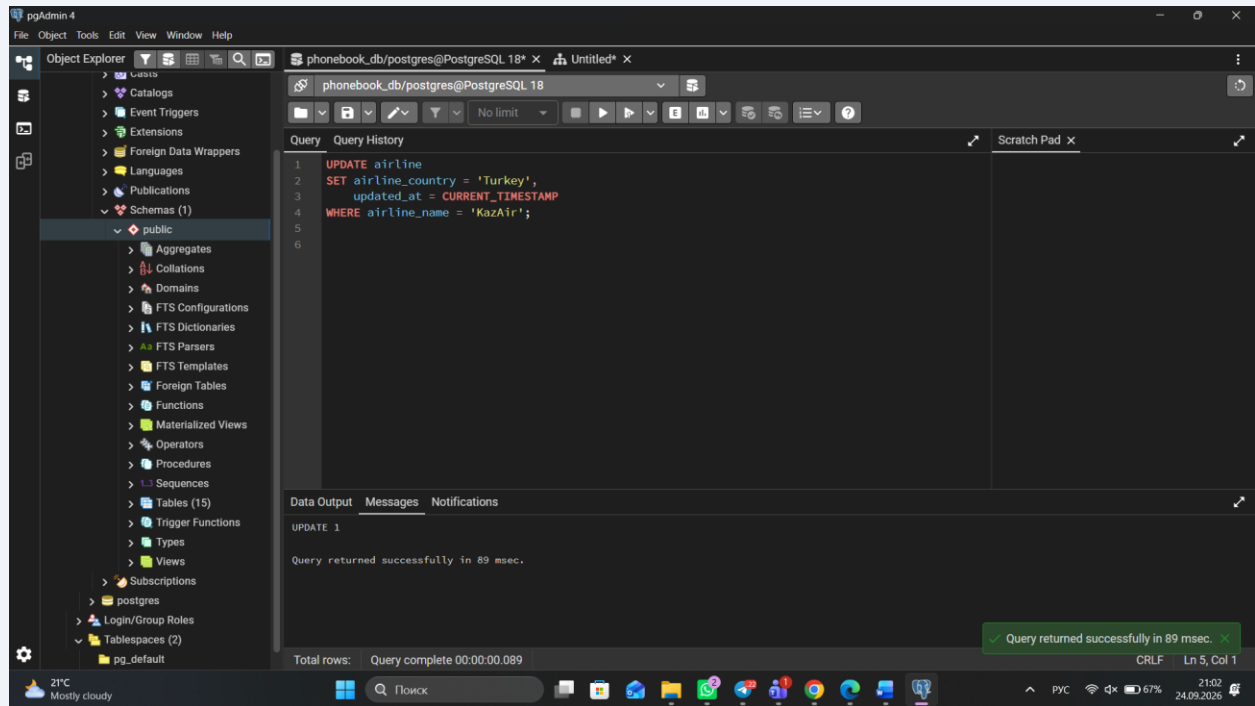
TASK2

Task 3: Update the country for 'KazAir'

Description: Change the country of 'KazAir' to 'Turkey'.

```
UPDATE airline
SET airline_country = 'Turkey',
    updated_at = CURRENT_TIMESTAMP
WHERE airline_name = 'KazAir';
```

Explanation: The UPDATE command searches for the row where the airline name is 'KazAir'. Then, it changes the country field to 'Turkey' and updates the modified time.



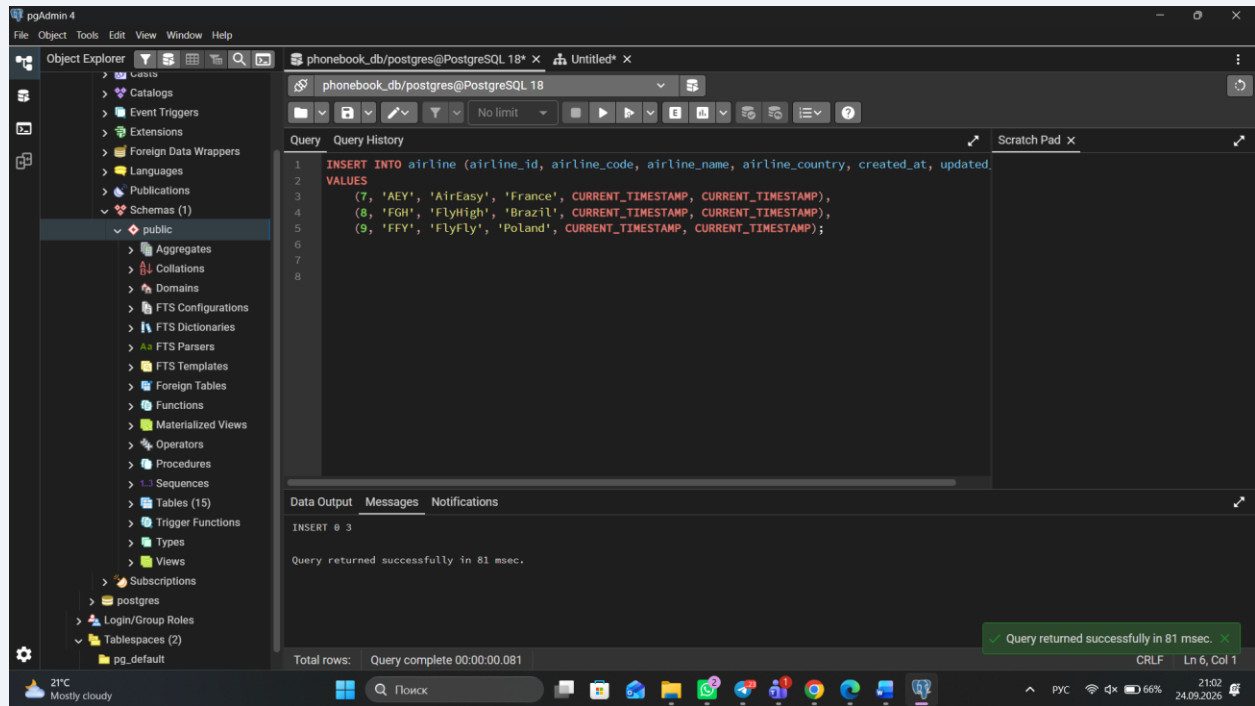
TASK3

Task 4: Add three airlines at once

Description: Insert three new airlines ('AirEasy', 'FlyHigh', and 'FlyFly') using a single SQL query.

```
INSERT INTO airline (airline_id, airline_code, airline_name, airline_country,
created_at, updated_at)
VALUES
(7, 'AEY', 'AirEasy', 'France', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
(8, 'FGH', 'FlyHigh', 'Brazil', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP),
(9, 'FFY', 'FlyFly', 'Poland', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP);
```

Explanation: We can add multiple rows at the same time by listing them in the VALUES section separated by commas. This is faster than writing three separate INSERT queries.



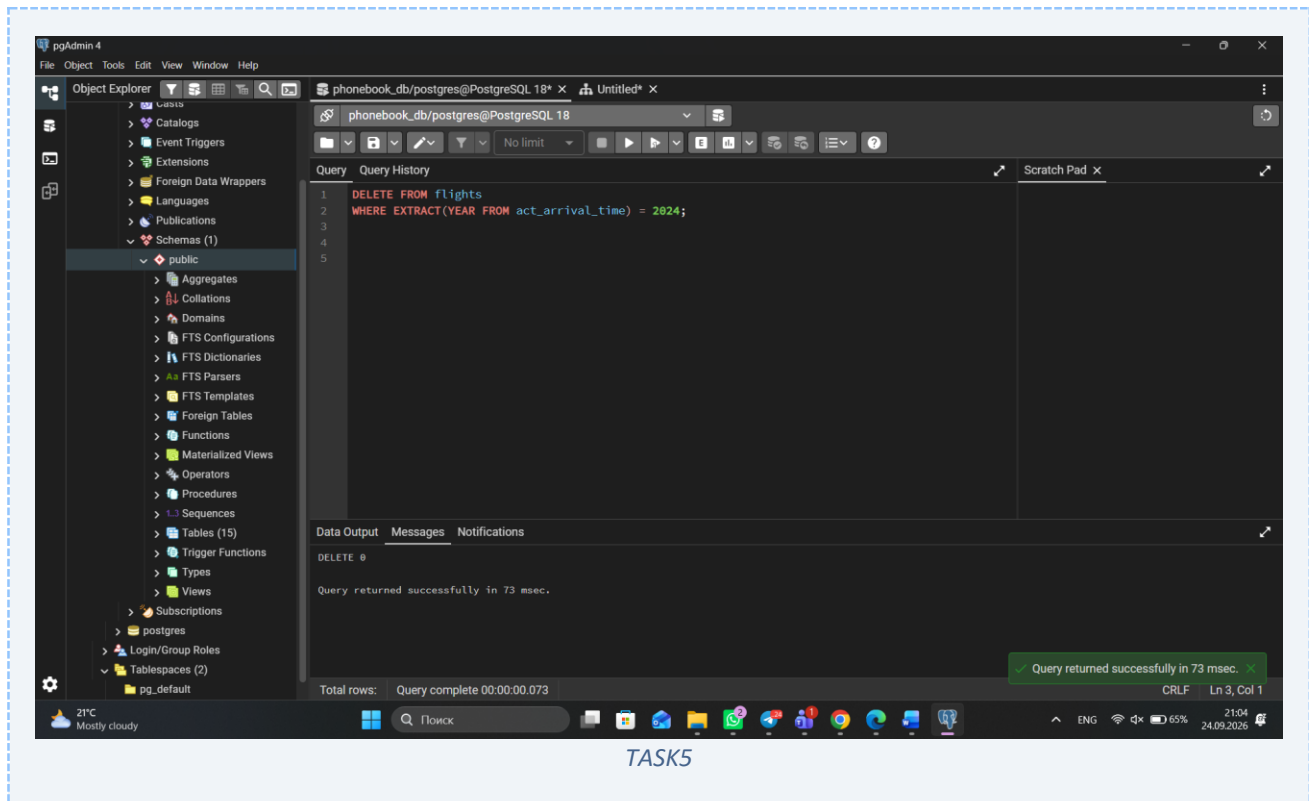
TASK4

Task 5: Delete flights arriving in 2024

Description: Remove all flights that are scheduled to arrive in the year 2024.

```
DELETE FROM flights
WHERE EXTRACT(YEAR FROM act_arrival_time) = 2024;
```

Explanation: The EXTRACT function pulls just the year out of the full arrival time. The DELETE command then removes any row where this year equals 2024.

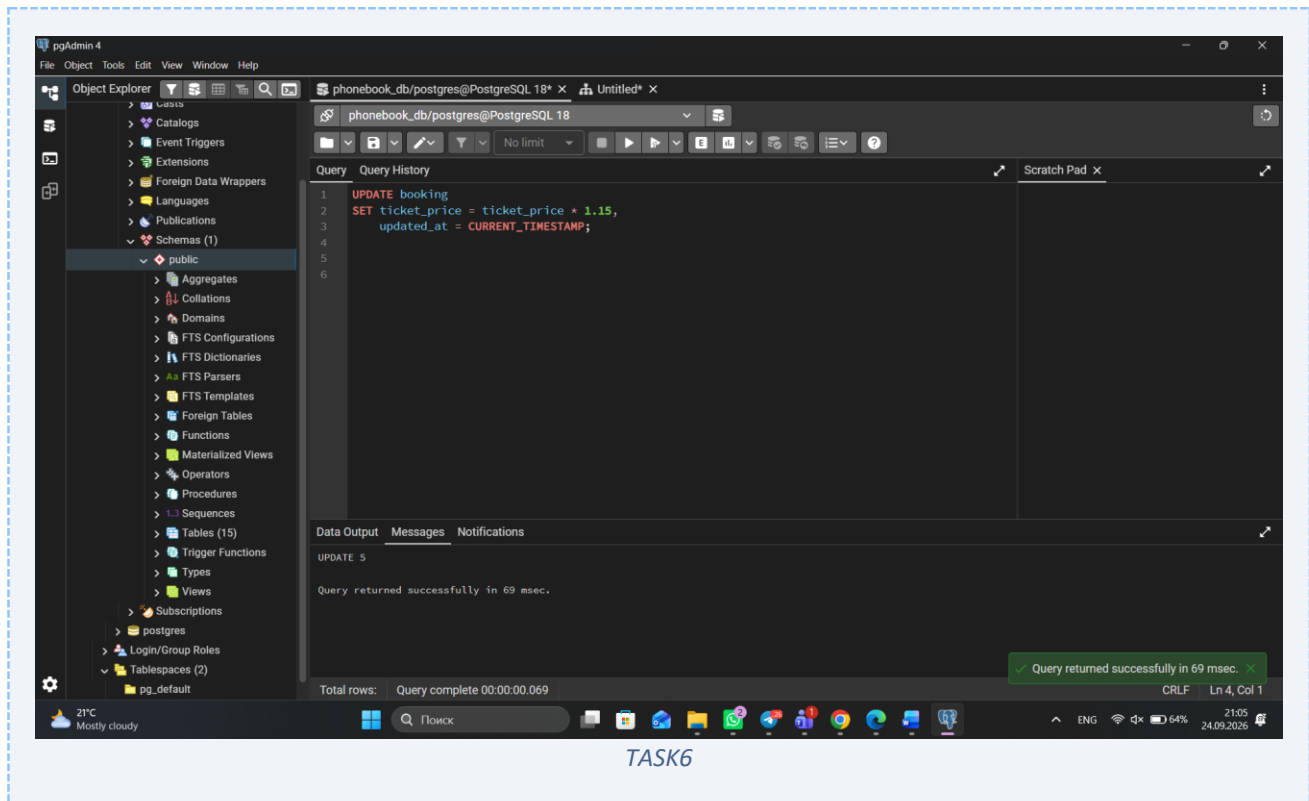


Task 6: Increase ticket prices by 15%

Description: Raise the ticket_price by 15% for every booking in the database.

```
UPDATE booking
SET ticket_price = ticket_price * 1.15,
    updated_at = CURRENT_TIMESTAMP;
```

Explanation: Because there is no WHERE clause in this query, it applies the math (price * 1.15) to every single row in the booking table.

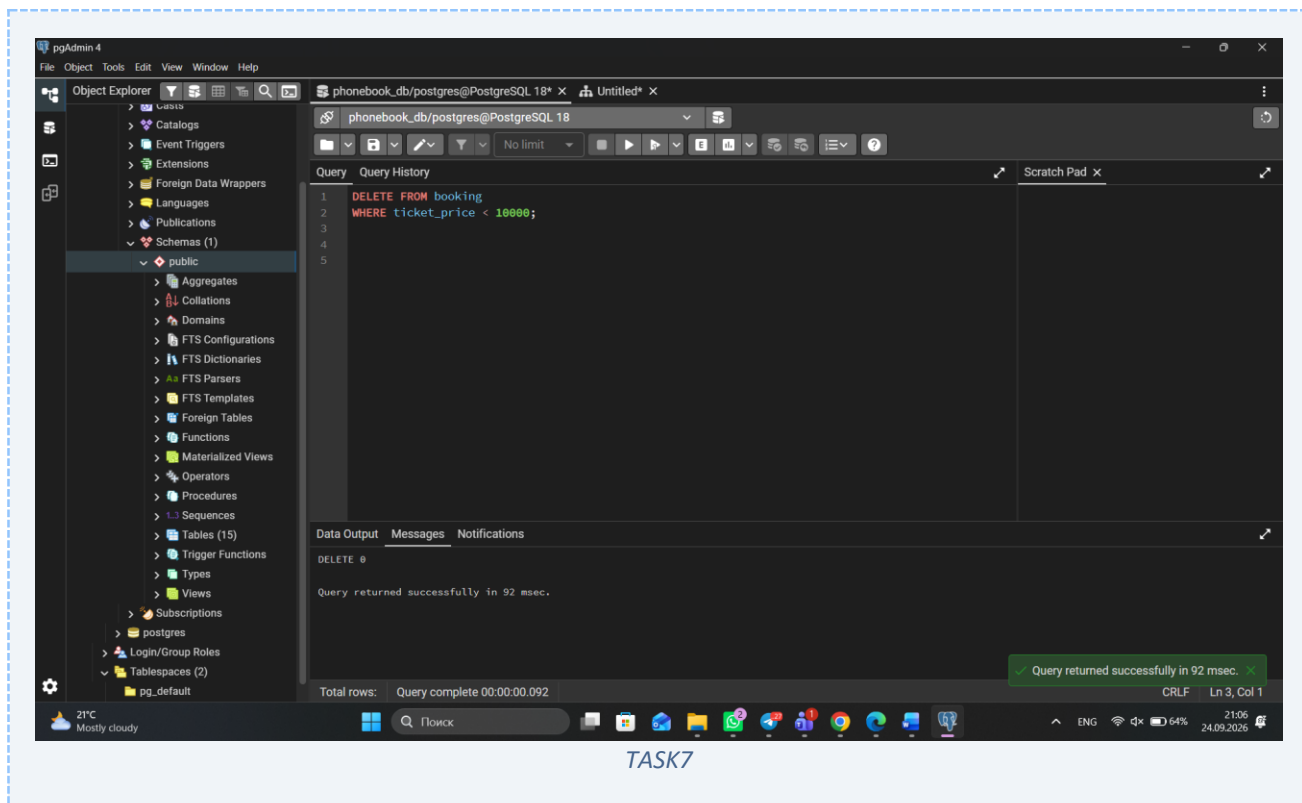


Task 7: Delete tickets priced under 10000

Description: Remove any booking where the ticket price is less than 10,000.

```
DELETE FROM booking
WHERE ticket_price < 10000;
```

Explanation: This simple DELETE statement looks for any row with a price lower than 10,000 and deletes it from the table permanently.

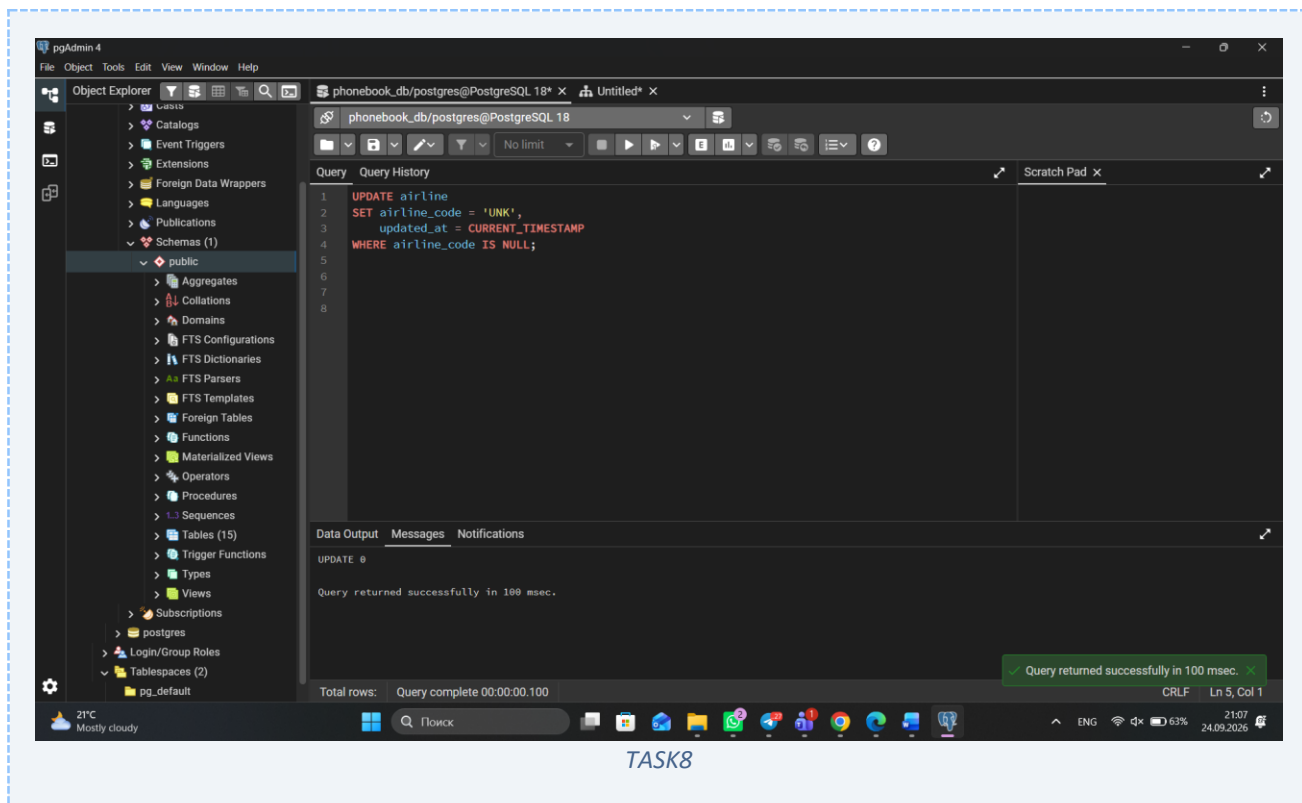


Task 8: Set default code 'UNK'

Description: If an airline_code is empty (NULL), change it to 'UNK' (Unknown).

```
UPDATE airline
SET airline_code = 'UNK',
    updated_at = CURRENT_TIMESTAMP
WHERE airline_code IS NULL;
```

Explanation: The IS NULL condition identifies rows that are missing an airline code. We use SET to fill those empty spots with the text 'UNK'.



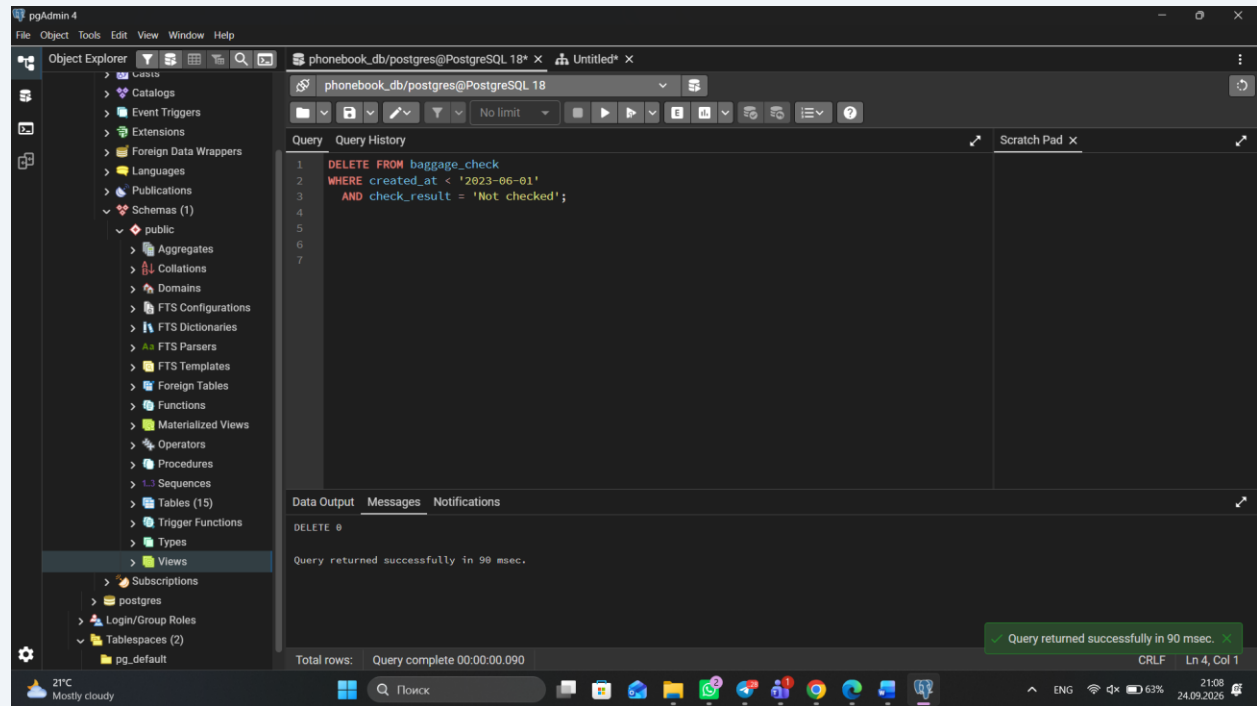
TASK8

Task 9: Delete old unchecked baggage

Description: Delete baggage checks created before June 1, 2023, if they still have a 'Not checked' status.

```
DELETE FROM baggage_check
WHERE created_at < '2023-06-01'
  AND check_result = 'Not checked';
```

Explanation: The AND keyword lets us check two rules at once: the date must be old (before June 2023), and the result must be 'Not checked'.



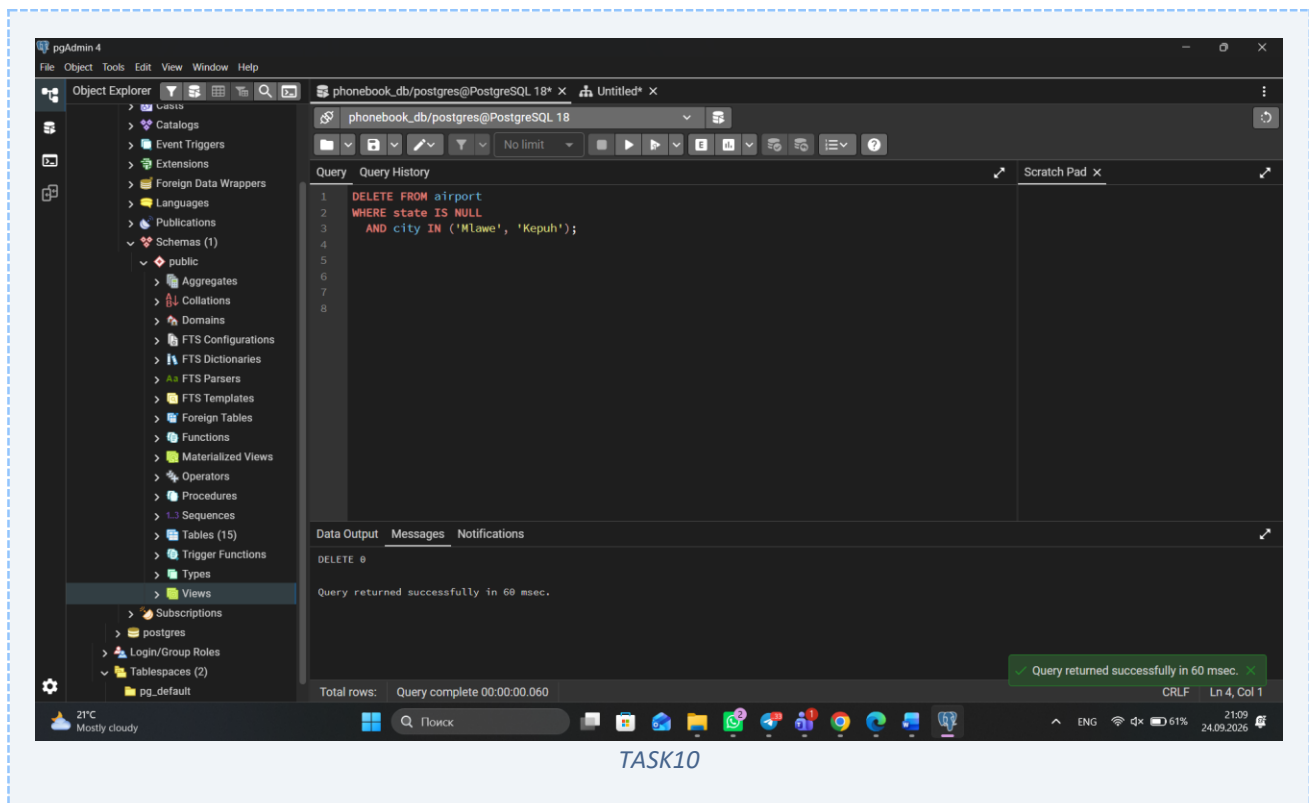
TASK9

Task 10: Delete incomplete airport data

Description: Remove airports located in 'Mlawe' or 'Kepuh' if their state field is empty (NULL).

```
DELETE FROM airport
WHERE state IS NULL
AND city IN ('Mlawe', 'Kepuh');
```

Explanation: We use the IN operator to check if the city matches either name in the list, and IS NULL to ensure the state information is missing.



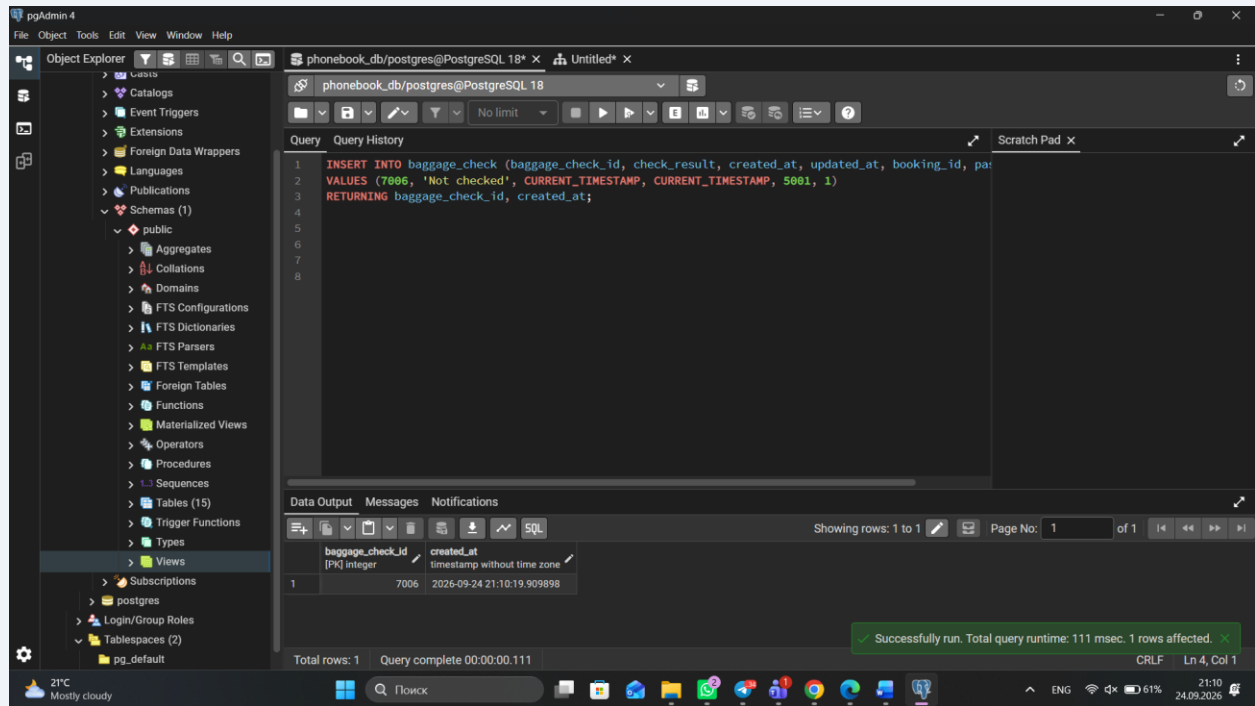
TASK10

Task 11: Insert baggage and return its ID

Description: Add a new unchecked baggage record and instantly view its generated ID and timestamp.

```
INSERT INTO baggage_check (baggage_check_id, check_result, created_at, updated_at,
booking_id, passenger_id)
VALUES (7006, 'Not checked', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP, 5001, 1)
RETURNING baggage_check_id, created_at;
```

Explanation: The RETURNING keyword forces PostgreSQL to output specific columns right after creating the row. This saves us from having to run a separate SELECT query.



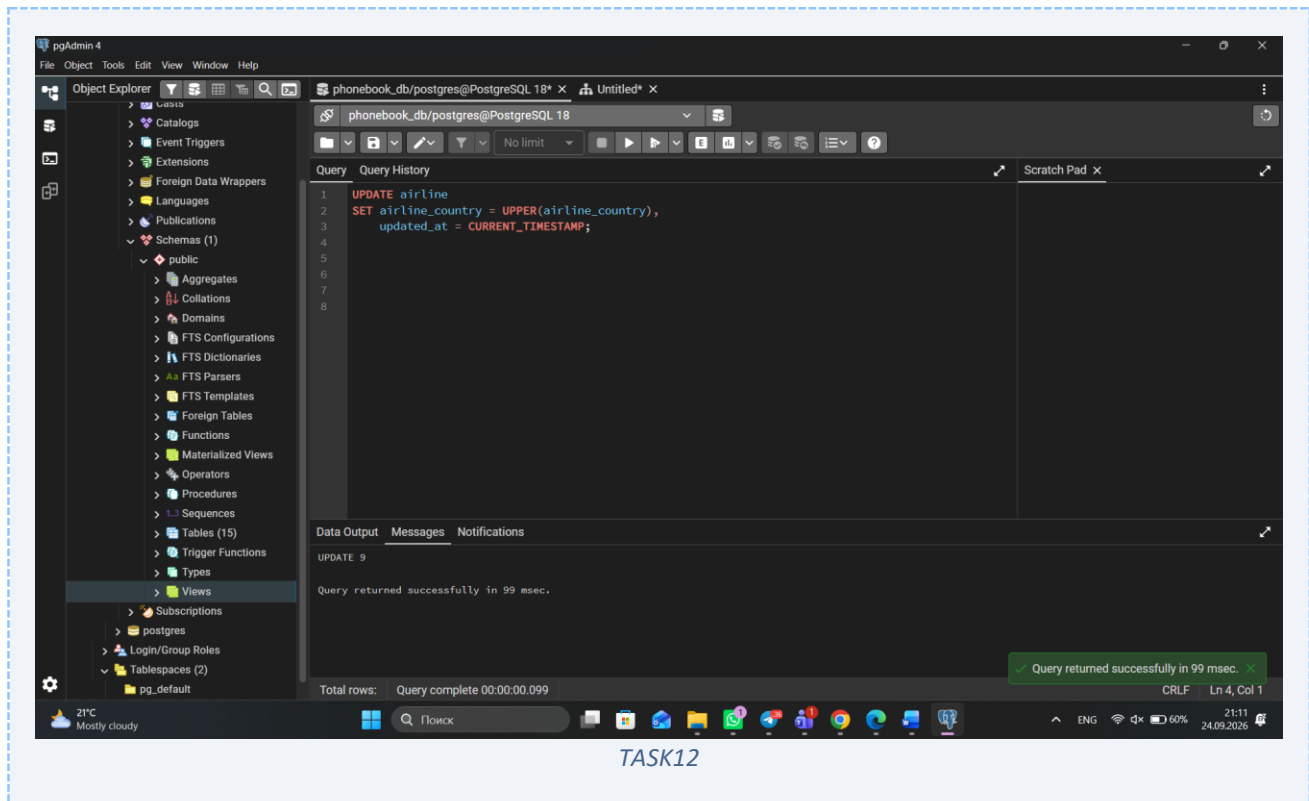
TASK11

Task 12: Uppercase all country names

Description: Change all airline country names to UPPERCASE letters to keep the data format clean.

```
UPDATE airline
SET airline_country = UPPER(airline_country),
    updated_at = CURRENT_TIMESTAMP;
```

Explanation: The UPPER() function takes the current string and capitalizes all of its letters (for example, 'Turkey' becomes 'TURKEY').



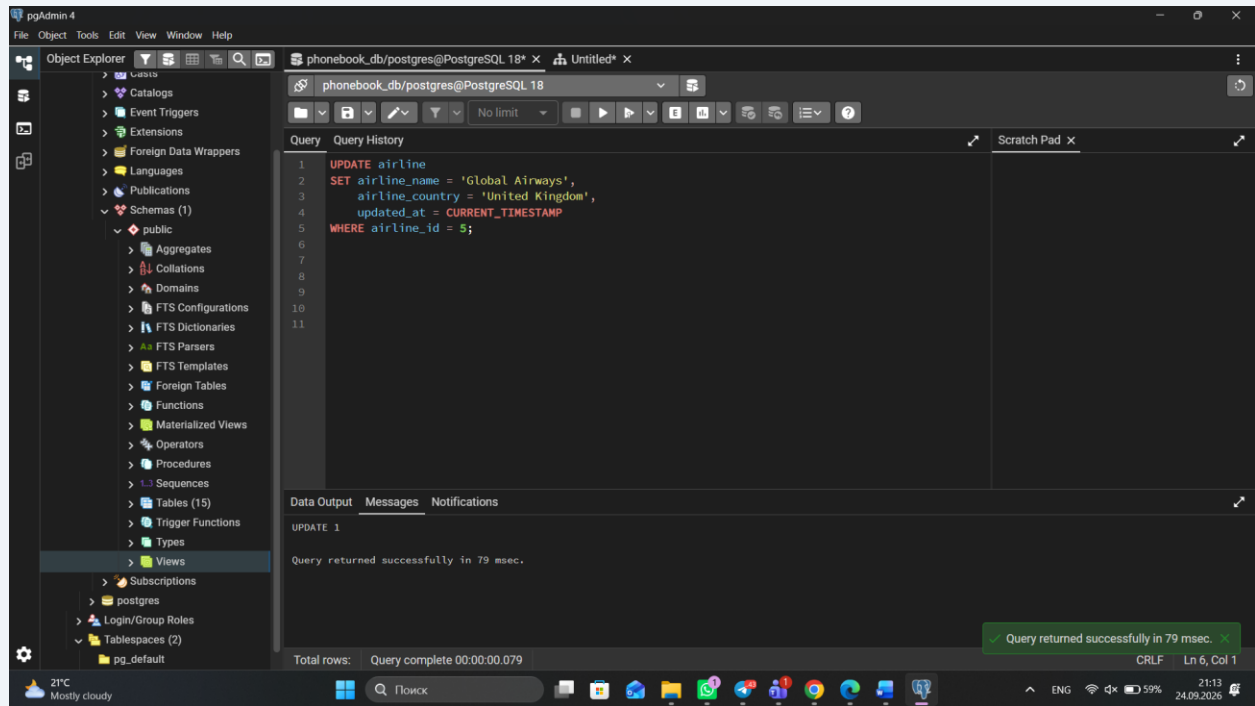
TASK12

Task 13: Update airline with ID 5

Description: Change the name to 'Global Airways' and country to 'United Kingdom' for the airline with ID 5.

```
UPDATE airline
SET airline_name = 'Global Airways',
    airline_country = 'United Kingdom',
    updated_at = CURRENT_TIMESTAMP
WHERE airline_id = 5;
```

Explanation: We use the primary key (airline_id = 5) to safely update exactly one row, changing both the name and the country at the same time.



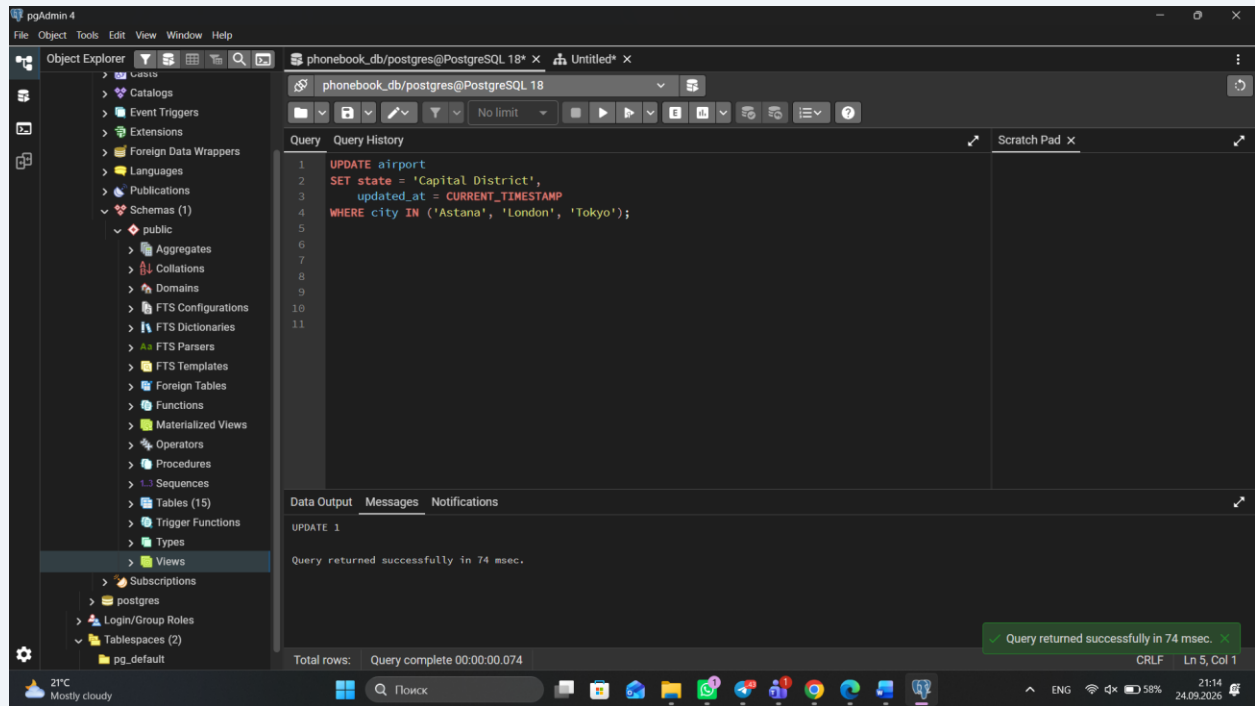
TASK13

Task 14: Set state to 'Capital District'

Description: Give the state name 'Capital District' to airports located in 'Astana', 'London', and 'Tokyo'.

```
UPDATE airport
SET state = 'Capital District',
    updated_at = CURRENT_TIMESTAMP
WHERE city IN ('Astana', 'London', 'Tokyo');
```

Explanation: The IN list quickly checks if an airport's city is one of these three capitals. If there is a match, it updates the state column.



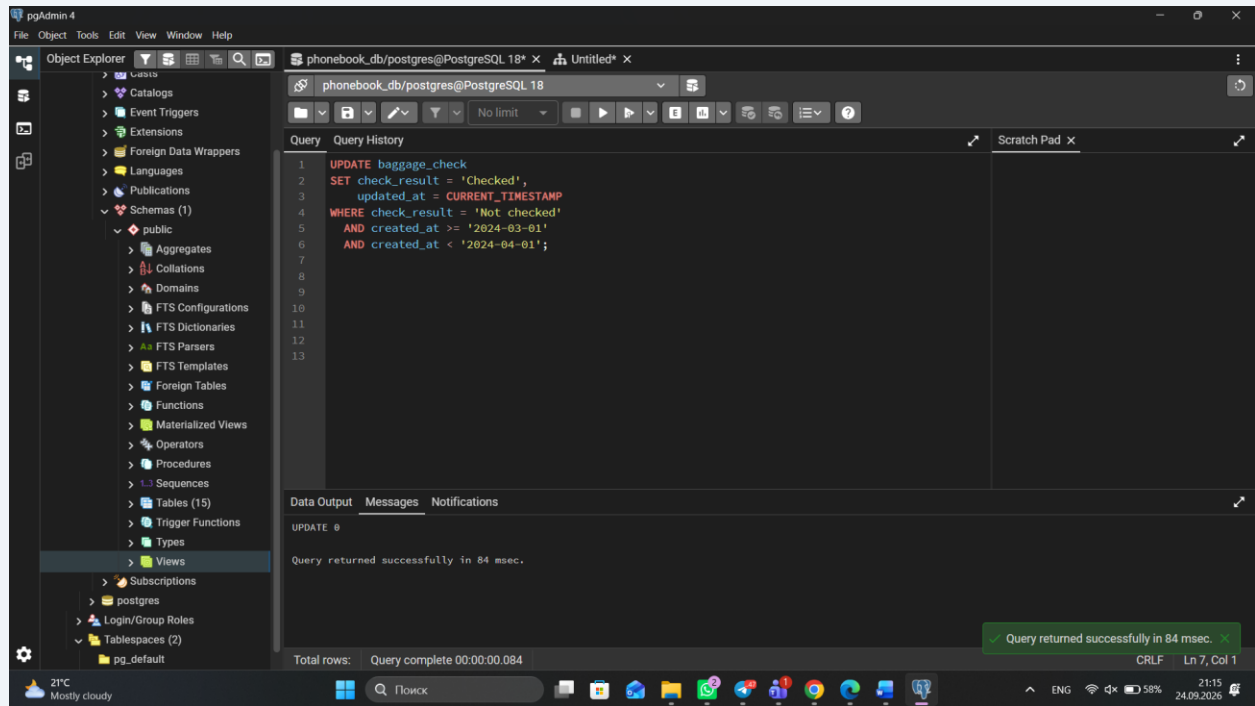
TASK14

Task 15: Mark March 2024 baggage as 'Checked'

Description: Change the status to 'Checked' for all unchecked baggage records from March 2024.

```
UPDATE baggage_check
SET check_result = 'Checked',
    updated_at = CURRENT_TIMESTAMP
WHERE check_result = 'Not checked'
    AND created_at >= '2024-03-01'
    AND created_at < '2024-04-01';
```

Explanation: By checking if the date is greater than or equal to March 1st and less than April 1st, we correctly target only the records made in March 2024.



TASK15

3. Conclusion

In this lab, we successfully practiced using Data Manipulation Language (DML) commands in PostgreSQL. We learned how to generate large amounts of data, insert single and multiple rows efficiently, update specific fields, and safely delete old or incomplete data. Understanding these commands is essential for keeping the database clean, accurate, and easy to manage.